

Risk Analysis – Trendora

Men's Clothing E-Commerce System

Ali Mohammed Al-Mezael | Ammar Hassan Al-Hashedi | Mohanad Luteify

UYG338 – Software Project Management

1. Risk Identification

Risk identification is the process of recognizing potential problems that may affect the development or operation of the Trendora Men's Clothing E-Commerce System. Possible risks in the project include server downtime that may prevent users from accessing the website, data loss caused by the in-memory database resetting on restart, authentication failures that may log users out during a session, team availability issues during the final presentation, and deployment failures when moving from a local environment to the cloud. Identifying these risks early helps the development team prepare solutions and preventive measures before they occur.

Table 1 below lists all identified risks with their descriptions and categories.

Table 1: Risk Identification

Risk ID	Risk Name	Description	Category
R1	Server Sleep	Render free hosting shuts down after 15 minutes of inactivity. First request after sleep takes up to 50 seconds.	Infrastructure
R2	Data Loss on Restart	Trendora uses an in-memory database (JavaScript arrays in db.js). All data is wiped when the server restarts.	Technical
R3	JWT Token Expiry	JWT tokens issued after login may expire during a long demo session, causing users to lose access.	Security
R4	Team Member Absent	One team member may be unable to attend the final presentation due to illness or emergency.	People
R5	GitHub Repo Private	The GitHub repository may accidentally be set to Private, blocking the professor from accessing the source code.	Process
R6	Browser Compatibility	CSS layout and styles may render incorrectly on browsers other than Chrome during the demo.	Technical

Risk ID	Risk Name	Description	Category
R7	Scope Creep	Team members may add extra features beyond the UML diagram, causing delays to required functionality.	Project
R8	API Deployment Failure	Frontend using hardcoded localhost:3000 in api.js fails when deployed to Render cloud servers.	Technical

1.1 Risk Assessment and Management

After identifying risks, each risk is evaluated based on its likelihood and impact. This evaluation helps prioritize which risks require the most attention and resources.

- Server Sleep (R1) – Likelihood: High | Impact: Medium | Management: Use UptimeRobot to keep the server active at all times.
- Data Loss on Restart (R2) – Likelihood: High | Impact: High | Management: Pre-load all essential data in db.js so it is always available after a restart.
- JWT Token Expiry (R3) – Likelihood: Low | Impact: Medium | Management: Set token expiry to 7 days in the authentication middleware.
- Team Member Absent (R4) – Likelihood: Medium | Impact: High | Management: All members learned the complete codebase and backup roles were assigned.
- GitHub Repo Private (R5) – Likelihood: Low | Impact: High | Management: Verify repository visibility before every submission deadline.
- Browser Compatibility (R6) – Likelihood: Low | Impact: Low | Management: Test on Chrome and Edge before every demo.
- Scope Creep (R7) – Likelihood: Medium | Impact: Medium | Management: Follow the UML class diagram strictly; no new features without team approval.
- API Deployment Failure (R8) – Likelihood: Medium | Impact: High | Management: Use relative /api paths in the frontend instead of hardcoded localhost addresses.

Risk management ensures that the Trendora system remains stable and accessible during development and during the final demonstration.

1.2 Mitigation / Preventive Actions

To reduce the impact of risks, preventive measures were applied throughout the development and deployment of Trendora. These measures include:

- Configuring UptimeRobot to send a ping request to the Render server every 5 minutes to prevent it from going to sleep.

- Pre-seeding the in-memory database in db.js with 10 products, one regular user account, and one admin account so that data is always available after any server restart.
- Setting the JWT token expiry to 7 days in middleware/auth.js to prevent session timeouts during demos or regular use.
- Ensuring all three team members understood the complete codebase, not only their assigned modules, so the presentation can continue if one member is unavailable.
- Changing all API calls in frontend/js/api.js from the hardcoded localhost address to a relative /api path, which works in both local and cloud environments.
- Testing the application in Chrome and Microsoft Edge before every session to catch any visual issues early.
- Agreeing to build only the features defined in the original UML class diagram and requiring team approval before any change to the project scope.

Applying these actions helps maintain the reliability, security, and smooth operation of the Trendora system.

1.3 Risk Matrix

Table 2 below shows the risk matrix for all identified risks, including their likelihood, impact, overall risk level, and the mitigation solution applied.

Table 2: Risk Matrix

Risk ID	Risk Description	Likelihood	Impact	Risk Level	Mitigation / Solution
R1	Server goes to sleep after 15 min of inactivity	High	Medium	High	Set up UptimeRobot to ping the server every 5 minutes and keep it active
R2	All data is wiped when the server restarts	High	High	Critical	Pre-load all demo data in db.js so it is available on every server start
R3	JWT token expires during the demo session	Low	Medium	Low	Set token expiry to 7 days in middleware/auth.js
R4	Team member absent during final presentation	Medium	High	High	All members studied the full codebase and backup roles were assigned
R5	GitHub repo set to Private before submission	Low	High	Medium	Check repo visibility before every deadline and keep a local ZIP backup
R6	CSS layout breaks on non-Chrome browsers	Low	Low	Low	Test on Chrome and Edge before every demo session

Risk ID	Risk Description	Likelihood	Impact	Risk Level	Mitigation / Solution
R7	Features added beyond the UML diagram scope	Medium	Medium	Medium	Strictly follow the UML class diagram; all changes require team approval
R8	API calls fail when deployed to cloud environment	Medium	High	High	Changed all API calls in api.js from localhost to relative /api path

Figure 1 below provides a visual representation of the risk matrix, showing where each risk is positioned based on its probability and impact level.

Figure 1: Risk Matrix – Probability vs Impact

Probability \ Impact	Low Impact	Medium Impact	High Impact
High Probability	Medium	High R1 – Server Sleep	Critical R2 – Data Loss
Medium Probability	Low	Medium R7 – Scope Creep	High R4 – Team Absent R8 – API Failure
Low Probability	Low R6 – Browser Issues	Low R3 – Token Expiry	Medium R5 – Repo Private

2. Code Security and Data Privacy

Security and privacy are essential in any e-commerce system because the platform handles sensitive customer information including names, email addresses, and passwords. The Trendora system was designed with several security measures to protect both the application and its users.

2.1 Code Security

To protect the Trendora system from potential attacks or vulnerabilities, the following security practices were implemented:

- **Authentication System:** A secure login system was implemented using JSON Web Tokens (JWT). After a successful login, the server generates a signed token that the user must include in every subsequent request. This ensures that only authenticated users can access protected routes.

- **Route Protection:** The `authMiddleware` function in `middleware/auth.js` verifies the JWT token before allowing access to protected endpoints such as `/api/cart` and `/api/orders`. Any request with a missing or invalid token receives a 401 Unauthorized response.
- **Admin Access Control:** The `adminMiddleware` function checks whether the decoded JWT token contains the `isAdmin` flag set to `true`. Regular users who attempt to access admin routes such as `/api/admin` receive a 403 Forbidden response.
- **Error Handling:** Error messages returned by the API are informative for the user but do not expose internal system details, file paths, or database structure information.
- **Input Validation:** User inputs such as email, password, and product details are validated on the server side before being processed to prevent unexpected behavior.

2.2 Data Privacy

Protecting user data is a fundamental aspect of the Trendora system. The following measures were applied to ensure data privacy:

- **Password Encryption:** All user passwords are hashed using the `bcrypt` library with 10 salt rounds before being stored in the database. Plain text passwords are never saved anywhere in the system.
- **Restricted Database Access:** Only the server-side code can interact with the database. No database credentials or connection details are exposed to the frontend or to the client browser.
- **No Sensitive Data in Frontend:** No passwords, secret keys, or tokens are stored in the frontend JavaScript files. The JWT token is stored in the browser's `localStorage` only after a successful login.
- **Minimal Data Collection:** Trendora only collects the information required to operate the system, including the user's name, email address, and hashed password.

3. Implementation Roadmap

The development of the Trendora Men's Clothing E-Commerce System followed several organized stages to ensure structured and reliable delivery.

1. Planning

Define project goals, user requirements, and team responsibilities. Review the UML class diagram and agree on the technology stack: Node.js, Express.js, HTML, CSS, and JavaScript.

2. System Design

Design the system architecture based on the 10 UML classes: User, Admin, Product, Category, ShoppingCart, CartItem, Order, OrderItem, Payment, and DatabaseConnection. Create the in-memory database structure in `db.js` and plan the REST API endpoints.

3. Development

Build the backend using Node.js and Express with five route files: auth.js, products.js, cart.js, orders.js, and admin.js. Build the frontend using HTML, CSS, and JavaScript with api.js handling all server communication. Implement JWT authentication and bcrypt password hashing.

4. Testing

Test all API endpoints using the browser and manual requests. Verify that user registration, login, product browsing, cart operations, and order placement all work correctly. Test on both localhost and the Render deployment.

5. Deployment

Deploy the application to Render cloud hosting. Push the source code to GitHub and connect the repository to Render. Verify that the live website is accessible at <https://trendora-ur4q.onrender.com>.

6. Maintenance

Monitor the live application for any issues. Fix bugs identified during testing or the final demo. Keep the GitHub repository updated with the latest version of the code.

Technologies Used:

- HTML / CSS: For designing the user interface and making it responsive across different screen sizes.
- JavaScript: For implementing frontend logic, form handling, and API communication.
- Node.js + Express.js: For building the backend REST API and handling server-side logic.
- bcryptjs: For hashing and verifying user passwords securely.
- jsonwebtoken (JWT): For generating and verifying authentication tokens.

4. Validation and Testing Plan

4.1 Testing Methods

Various testing approaches were used to ensure the quality and reliability of the Trendora system:

- Unit Testing: Each individual function and module was tested separately. This includes testing the register function, login function, add to cart function, and place order function to confirm they return the correct results.

- **Integration Testing:** The interaction between different modules was tested to ensure they work correctly together. For example, after a user logs in, the cart routes must correctly identify the user and return their cart data.
- **System Testing:** The complete application was tested as a whole, including the full flow of browsing products, adding items to the cart, and placing an order from the frontend to the backend.
- **User Acceptance Testing:** The live version of the application at <https://trendora-ur4q.onrender.com> was tested to confirm that all features work correctly in the production environment.

4.2 Validation Methods

Validation checks were performed to confirm that the system behaves correctly and meets the project requirements:

- **Functional Validation:** All features defined in the UML class diagram were implemented and verified to work as intended, including all CRUD operations for products, cart, and orders.
- **Data Validation:** User input such as email format, required fields, and numeric prices were validated on the server side before being processed or stored.
- **Security Validation:** Protected routes were tested with missing tokens, expired tokens, and non-admin tokens to confirm that the middleware correctly rejects unauthorized requests.
- **Usability Validation:** The frontend interface was reviewed to ensure that navigation, product filtering, cart operations, and checkout are clear and easy to use.

4.3 Performance Evaluation

The performance of the Trendora system was evaluated using the following criteria:

- **Page Load Time:** The homepage with 10 product cards loads within 2 seconds on a standard connection when the server is active.
- **System Stability:** The application ran without crashes during all test sessions. No memory issues were observed during normal usage.
- **API Response Speed:** All API endpoints respond in under 300 milliseconds for standard requests on the Render deployment.
- **User Experience:** Test users were able to complete the full shopping flow, from browsing products to placing an order, without confusion or errors.

5. Success Criteria

The Trendora project is considered successful when it meets the following criteria:

- Users can register a new account and log in securely using email and password.
- Users can browse the full product catalog and filter products by category.
- Users can add products to their shopping cart, update quantities, and remove items.
- Users can place an order by entering a shipping address and selecting a payment method.
- Administrators can log in to the admin dashboard and view all orders, users, and system statistics.
- All sensitive data including passwords is encrypted and never exposed to the frontend.
- The application is deployed and accessible to anyone via the public Render URL.
- The source code is publicly available on GitHub for review.

If these conditions are met, the Trendora project will be considered a complete and successful implementation of a men's clothing e-commerce system.